

Лабораторная работа № 5

Вероятностные алгоритмы проверки чисел на простоту

Юрченко Артём Алексеевич

Содержание I

- 1 Введение
- 2 тест Ферма
- 3 Алгоритм символа Якоби
- 4 Алгоритм Соловья-Штрассена
- 5 Алгоритм Миллера-Рабина

Содержание II

6 Итоги

Section 1

Введение

Введение

В данном отчёте будет представлена реализация алгоритмов проверки простоты числа.

Основные алгоритмы

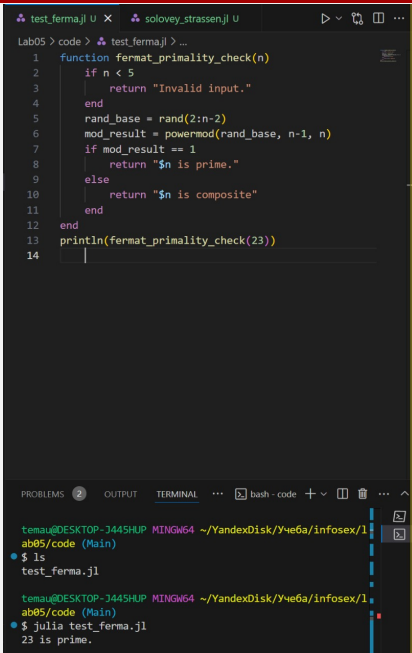
- тест Ферма
- Алгоритм символа Якоби
- тест Соловья-Штрассена
- тест Миллера-Рабина

Section 2

тест Ферма

Реализация тест Ферма

Тест Ферма основан на критерии Эйлера.



The image shows a code editor with two tabs: 'test_ferma.jl U' and 'solovey_strassen.jl U'. The 'test_ferma.jl' tab is active, displaying a Julia script. The script defines a function 'fermat_primality_check(n)' that checks if a number 'n' is prime using Fermat's little theorem. It returns 'Invalid input.' for $n < 5$, '23 is prime.' for $n = 23$, and ' n is composite' for other values. The script then calls 'println(fermat_primality_check(23))' and prints the output.

```
1 function fermat_primality_check(n)
2     if n < 5
3         return "Invalid input."
4     end
5     rand_base = rand(2:n-2)
6     mod_result = powermod(rand_base, n-1, n)
7     if mod_result == 1
8         return "$n is prime."
9     else
10        return "$n is composite"
11    end
12 end
13 println(fermat_primality_check(23))
14
```

Below the code editor is a terminal window. It shows the command prompt 'temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1 ab05/code (Main)' and the execution of 'ls' and 'julia test_ferma.jl'. The output of the Julia script is '23 is prime.'

```
temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1
ab05/code (Main)
$ ls
test_ferma.jl

temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1
ab05/code (Main)
$ julia test_ferma.jl
23 is prime.
```

Section 3

Алгоритм символа Якоби

Реализация Алгоритм символа Якоби

В отличие от теста Ферма, алгоритм Якоби точнее, он основан на критерии Эйлера, который использует символ Якоби.

```

_ferma.jl U  solovey_strassen.jl U  jacobi.jl U x  ▸ ▾ 🔍 □ ...
Lab05 > code > solovey_strassen.jl > ...
1  function legendre_symbol(x, y)
5      symbol_result = 1
6      while x != 0
7          power_count = 0
8          while x % 2 == 0
9              power_count += 1
10             x ÷= 2
11         end
12
13         if power_count % 2 == 1 && (y % 8 == 3 || y
14             symbol_result *= -1
15         end
16
17         if y % 4 == 3 && x % 4 == 3
18             symbol_result *= -1
19         end
20
21         x, y = y % x, x
22     end
23     return y == 1 ? symbol_result : 0
24 end
25
26 println(legendre_symbol(23, 32))

```

PROBLEMS 2 TERMINAL ...

```

$ julia solovey_strassen.jl
123456 is composite.

temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Yче6a/infosex/1
ab05/code (Main)
$ julia jacobi.jl
1

```

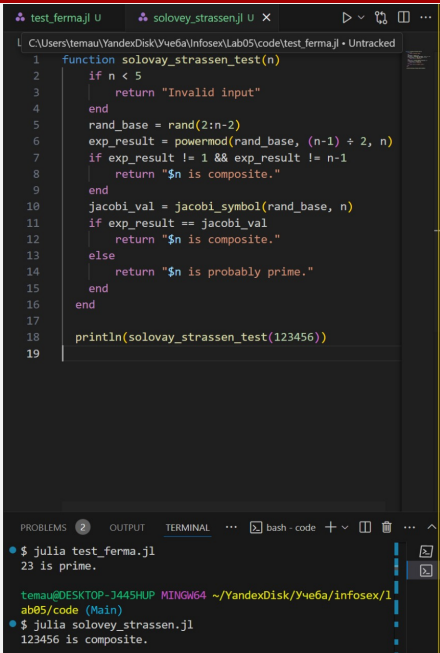
Section 4

Алгоритм Соловья-Штрассена

Реализация Алгоритма Соловья-Штрассена

Программная реализация алгоритма, реализующего
тест Соловья-Штрассена

Самый часто используемый алгоритм проверки числа
на простоту.



The screenshot shows a Julia REPL window with two tabs: `test_ferma.jl` and `solovey_strassen.jl`. The active tab is `solovey_strassen.jl`, which contains a function `solovay_strassen_test(n)`. The function implements the Solovay-Strassen primality test. It first checks if `n < 5`; if so, it returns "Invalid input". Then, it generates a random base `rand_base` between 2 and `n-2`. It calculates `exp_result = powermod(rand_base, (n-1) ÷ 2, n)`. If `exp_result != 1` and `exp_result != n-1`, it returns "`n` is composite.". Otherwise, it calculates the Jacobi symbol `jacobi_val = jacobi_symbol(rand_base, n)`. If `exp_result == jacobi_val`, it returns "`n` is composite.". Else, it returns "`n` is probably prime.". The function is called with `println(solovay_strassen_test(123456))`.

The bottom panel shows the terminal output. The first command is `julia test_ferma.jl`, which outputs `23 is prime.`. The second command is `julia solovey_strassen.jl`, which outputs `123456 is composite.`.

```
function solovay_strassen_test(n)
    if n < 5
        return "Invalid input"
    end
    rand_base = rand(2:n-2)
    exp_result = powermod(rand_base, (n-1) ÷ 2, n)
    if exp_result != 1 && exp_result != n-1
        return "$n is composite."
    end
    jacobi_val = jacobi_symbol(rand_base, n)
    if exp_result == jacobi_val
        return "$n is composite."
    else
        return "$n is probably prime."
    end
end

println(solovay_strassen_test(123456))
```

```
$ julia test_ferma.jl
23 is prime.

temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1
ab05/code (Main)
$ julia solovey_strassen.jl
123456 is composite.
```

Section 5

Алгоритм Миллера-Рабина

Реализация Алгоритма Миллера-Рабина

Тест Миллера-Рабина – это стохастический алгоритм проверки чисел на простоту. Он основан на свойствах модульной арифметики и является усовершенствованной версией теста Ферма.

The screenshot shows a Julia REPL session with two tabs: `test_ferma.jl` and `solovey_strassen.jl`. The active tab is `solovey_strassen.jl`, which contains the following code:

```

1 function solovay_strassen_test(n)
2     if n < 5
3         return "Invalid input"
4     end
5     rand_base = rand(2:n-2)
6     exp_result = powermod(rand_base, (n-1) ÷ 2, n)
7     if exp_result != 1 && exp_result != n-1
8         return "$n is composite."
9     end
10    jacobi_val = jacobi_symbol(rand_base, n)
11    if exp_result == jacobi_val
12        return "$n is composite."
13    else
14        return "$n is probably prime."
15    end
16 end
17
18 println(solovay_strassen_test(123456))
19

```

The bottom panel shows the terminal output:

```

$ julia test_ferma.jl
23 is prime.

temau@DESKTOP-J445HUP MINGW64 ~/YandexDisk/Учеба/infosex/1
ab05/code (Main)
$ julia solovey_strassen.jl
123456 is composite.

```

Section 6

Итоги

Заключение

В данной лабораторной работе были реализованы шифры алгоритмы проверки числа на простоту.